

DEEP LEARNING DOCUMENTATION SERIES

Tensor

The fundamental building block of everything in PyTorch. Creating, manipulating, reshaping, and unleashing tensors on GPU.

Day

02 of ∞

Date

July 13, 2026

Topic

Tensors & Operations



OPENING

What Exactly Is a Tensor?

The word "tensor" sounds intimidating. It conjures images of Einstein's field equations and graduate-level physics. But in PyTorch, a tensor is beautifully simple — and understanding it deeply is the single most important thing you'll do in your deep learning journey.

*Imagine you're building a house. Bricks are the most fundamental unit — everything is made from them. In PyTorch, **tensors are your bricks**. Every image, every word embedding, every model weight, every gradient, every prediction — they're all tensors. If you understand tensors deeply, you understand PyTorch deeply. It's that simple.*

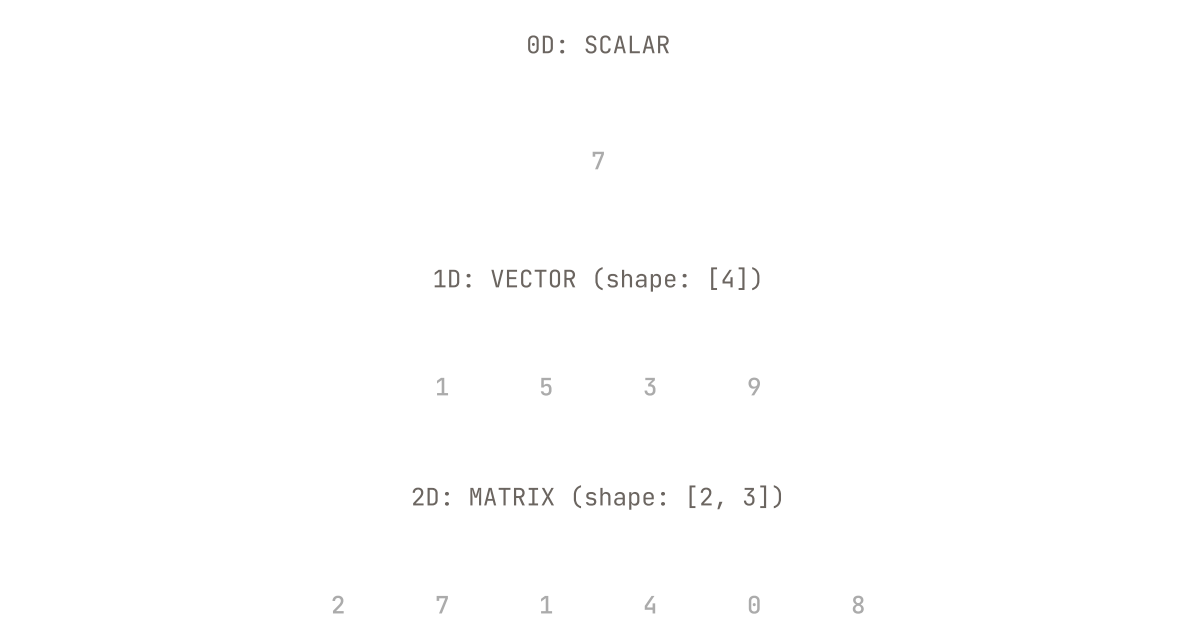
The Formal Definition

A **tensor** is an n-dimensional array of numbers. That's it. It generalizes the concepts you already know:

MATH CONCEPT	PROGRAMMING EQUIVALENT	PYTORCH NAME	DIMENSIONS	EXAMPLE
Scalar	A single number	0D Tensor	0	<code>tensor(5.0)</code>
Vector	A 1D array	1D Tensor	1	<code>tensor([1,2,3])</code>
Matrix	A 2D array	2D Tensor	2	<code>tensor([[1,2], [3,4]])</code>
3D Array	A 3D array	3D Tensor	3	A color image (H x W x C)

MATH CONCEPT	PROGRAMMING EQUIVALENT	PYTORCH NAME	DIMENSIONS	EXAMPLE
n-D Array	An nD array	nD Tensor	n	A batch of images (B x C x H x W)

Visualizing Tensor Dimensions



Key Insight

A PyTorch Tensor is not just a NumPy array with a new name. It has three superpowers that NumPy arrays lack: (1) GPU acceleration, (2) automatic differentiation (autograd), and (3) deep neural network integration. We'll explore all three as we go.

Tensor vs. NumPy Array

FEATURE	NUMPY NDARRAY	PYTORCH TENSOR
GPU Support	No	Yes (CUDA, MPS, ROCm)
Automatic Gradients	No	Yes (autograd)
Optimized for Deep Learning	No	Yes

FEATURE	NUMPY NDARRAY	PYTORCH TENSOR
Interoperability	Universal	Converts to/from NumPy easily

01

Creating Tensors

Every PyTorch journey begins with creating tensors. There are many ways to create them, each suited for different situations. Let's explore every method.

*Think of tensor creation methods as different tools in a workshop. You **could** use a hammer for everything, but a screwdriver works better for screws. Similarly, `torch.zeros()` is perfect for initializing placeholder arrays, while `torch.tensor()` is ideal for creating tensors from data you already have. Knowing all the tools makes you a craftsperson, not just a user.*

Quick Reference: All Creation Methods

METHOD	DESCRIPTION	USE CASE
<code>torch.empty(n,m)</code>	Uninitialized tensor	When you'll fill it immediately
<code>torch.zeros(n,m)</code>	All zeros	Bias init, accumulators
<code>torch.ones(n,m)</code>	All ones	Scaling factors, masks
<code>torch.rand(n,m)</code>	Uniform random [0,1]	Weight initialization
<code>torch.randn(n,m)</code>	Normal random (mean=0, std=1)	Weight initialization
<code>torch.tensor(data)</code>	From Python data	Loading your own data
<code>torch.arange(s,e,step)</code>	Range of values	Index sequences
<code>torch.linspace(s,e,n)</code>	Evenly spaced values	Smooth ranges, plotting
<code>torch.eye(n)</code>	Identity matrix	Linear algebra operations

METHOD	DESCRIPTION	USE CASE
<code>torch.full(shape, val)</code>	Fill with specific value	Constant tensors

Basic Arithmetic Operations

Once you have tensors, the next step is doing math with them. PyTorch makes arithmetic feel identical to Python — because it IS Python, just with tensors instead of numbers.

*Here's the beautiful thing about PyTorch tensors: every arithmetic operation you can do with numbers, you can do with tensors — and it happens **element-wise**. Multiply a tensor by 3, and every single element gets multiplied by 3. It's like having a spreadsheet where you apply a formula to every cell simultaneously.*

Element-Wise Operations on Two Tensors

When you have two tensors of the same shape, you can perform operations between them element by element. The element at position `[0][0]` in tensor A operates with the element at `[0][0]` in tensor B, and so on.

a * b (Element-wise)

Each element multiplies with its corresponding element

```
[[a11*b11, a12*b12],  
 [a21*b21, a22*b22]]
```

VS

a @ b (Matrix Multiplication)

Row-column dot products (linear algebra)

```
[[a11*b11+a12*b21, ...],  
 [a21*b11+a22*b21, ...]]
```

Useful Tensor Functions

PyTorch provides a rich library of functions that transform tensors in useful ways — from basic math operations to clamping values within ranges.

Quick Summary

FUNCTION	WHAT IT DOES	EXAMPLE	RESULT
<code>torch.abs(x)</code>	Remove negative signs	<code>abs([-1, 2, -3])</code>	<code>[1, 2, 3]</code>
<code>torch.neg(x)</code>	Flip all signs	<code>neg([1, -2, 3])</code>	<code>[-1, 2, -3]</code>
<code>torch.round(x)</code>	Nearest integer	<code>round([1.3, 3.7])</code>	<code>[1., 4.]</code>
<code>torch.ceil(x)</code>	Round up	<code>ceil([1.1, 3.01])</code>	<code>[2., 4.]</code>
<code>torch.floor(x)</code>	Round down	<code>floor([1.9, 3.99])</code>	<code>[1., 3.]</code>
<code>torch.clamp(x,min,max)</code>	Restrict to range	<code>clamp([0,5,10],2,8)</code>	<code>[2, 5, 8]</code>

Reduction Functions

Reduction functions take a tensor and **reduce** it to a smaller tensor (often a single value) by aggregating across dimensions. These are the backbone of computing loss, accuracy, and statistical summaries.

*When training a neural network, your model makes predictions for a batch of 64 images. You need to know: "How wrong are we, on average?" That requires **reducing** 64 individual error values into a single number — the loss. That's what reduction functions do. They compress information.*

Understanding Dimensions (dim) Visually

```
tensor([[7, 6, 9],      ← row 0 (sum across = 22)
        [6, 3, 1]])    ← row 1 (sum across = 10)
      ↑ ↑ ↑
      col0 col1 col2
(sum   (sum (sum
down=  down= down=
13)    9)   10)
```

dim=0: collapses rows → result shape: [3] (one value per column)

dim=1: collapses columns → result shape: [2] (one value per row)

Matrix Operations

Matrix operations are the mathematical heart of neural networks. Every layer in a neural network performs a matrix multiplication. Understanding these operations is non-negotiable.

*When you feed a batch of 64 images (each with 784 pixels) into a neural network layer with 128 neurons, what happens? A **matrix multiplication**: $[64 \times 784] @ [784 \times 128] = [64 \times 128]$. This single operation — repeated millions of times — is how neural networks learn. It's the engine of intelligence.*

Comparison Operations

Comparison operations return boolean tensors (True/False). They're essential for masking, filtering, and building conditional logic in your training pipeline.

Special Functions

Mathematical functions that are essential for neural networks — especially activation functions like sigmoid and softmax.

Inplace Operations

By default, operations create new tensors (new memory). Inplace operations modify the tensor directly, saving memory.

Normal Operation

```
# Creates NEW tensor (uses extra memory)  
result = m + n  
# m and n unchanged  
# "result" is a brand new tensor
```

VS

Inplace Operation

```
# Modifies m IN PLACE (no extra memory)  
m.add_(n)  
# m is now changed directly  
# The underscore "_" means inplace
```

Convention

Any PyTorch function ending with an underscore (`_`) modifies the tensor inplace.
Examples: `add_()` , `mul_()` , `zero_()` , `relu_()` . Use them to save memory,
but be careful — you lose the original values.

Copying Tensors

Copying seems simple, but PyTorch has a subtle trap that catches beginners off guard. Understanding it will save you hours of debugging.

Tensor Operations on GPU

This is where PyTorch truly shines. Moving tensors to GPU is a single line of code — and the speed difference is extraordinary.

*A CPU is like a brilliant professor — capable of complex sequential tasks, but working on one thing at a time. A GPU is like a stadium of 5,000 students — each one simple, but working **simultaneously**. For the matrix multiplications that power neural networks, that parallelism changes everything.*

The Speed Difference — Real Benchmark

Reshaping Tensors

In deep learning, you constantly need to change the shape of tensors without changing the data. An image might need to go from 3D to 1D before entering a fully connected layer. A batch might need an extra dimension for processing. Reshaping makes this possible.

Quick Reference: Reshaping Operations

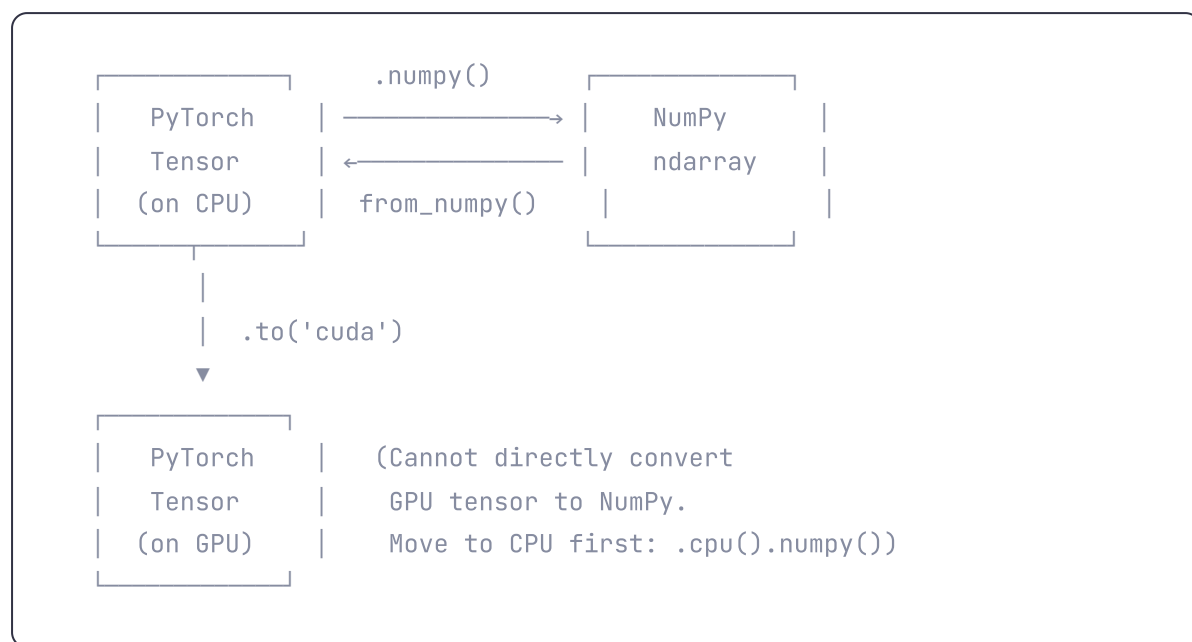
OPERATION	PURPOSE	SHAPE CHANGE
<code>reshape(a, b)</code>	Change shape	$[m*n] \rightarrow [m, n]$
<code>flatten()</code>	Collapse to 1D	$[m, n] \rightarrow [m*n]$
<code>permute(dims)</code>	Reorder dimensions	$[2,3,4] \rightarrow [4,2,3]$

OPERATION	PURPOSE	SHAPE CHANGE
<code>unsqueeze(dim)</code>	Add size-1 dimension	$[3,4] \rightarrow [1,3,4]$
<code>squeeze(dim)</code>	Remove size-1 dimension	$[1,20] \rightarrow [20]$

NumPy & PyTorch Conversions

PyTorch and NumPy are best friends. Converting between them is seamless and fast — they even share memory in many cases.

The Complete Conversion Map



SUMMARY

Everything We Covered Today

#	TOPIC	KEY TAKEAWAY
1	What is a Tensor	An n-dimensional array — the fundamental data structure of PyTorch
2	Creating Tensors	10+ methods: zeros, ones, rand, tensor, arange, linspace, eye, full
3	Manual Seed	<code>torch.manual_seed(42)</code> for reproducible experiments
4	Arithmetic	Scalar operations (element-wise): <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>//</code> , <code>%</code> , <code>**</code>
5	Element-wise Ops	Two tensors: operations happen position by position
6	Useful Functions	<code>abs</code> , <code>neg</code> , <code>round</code> , <code>ceil</code> , <code>floor</code> , <code>clamp</code>
7	Reduction	<code>sum</code> , <code>mean</code> , <code>median</code> , <code>max</code> , <code>min</code> , <code>prod</code> , <code>std</code> , <code>var</code> , <code>argmax</code> , <code>argmin</code>

#	TOPIC	KEY TAKEAWAY
8	Matrix Ops	matmul, dot, transpose, det, inverse
9	Comparison	>, <, ==, >=, <= — returns boolean tensors
10	Special Functions	log, exp, sqrt, sigmoid — activation functions
11	Inplace Ops	Functions ending with _ modify tensor in place (save memory)
12	Copying	= is a reference, .clone() is a true copy
13	GPU Operations	.to('cuda') — 696x speedup on matrix operations
14	Reshaping	reshape, flatten, permute, unsqueeze, squeeze
15	NumPy Conversion	.numpy() and torch.from_numpy()

“

You now understand the language that PyTorch speaks. Tensors are its words. Operations are its grammar. With what you learned today, you can express any computation that a neural network needs.

Tomorrow, we use these building blocks to construct our first neural network.

— Day 2 Complete

